

Property Function Decorator : Code-Along (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-v2-property-decorator>  <https://github.com/learn-co-curriculum/python-p3-v2-property-decorator/issues/new>

Learning Goals

- Use `property()` as a Decorator

Key Vocab

- **Attribute**: variables that belong to an object.
- **Property**: attributes that are controlled by methods.
- **Decorator**: function that takes another function as an argument and returns a new function with added functionality.

Introduction

Properties in Python are attributes that are controlled by methods. We've already seen how to use the `property()` function to define getter and setter methods that control how object attributes are accessed and modified.

A **decorator** is a function that takes another function as an argument and returns a new function, often attaching pre- or post-processing functionality. The decorator syntax was not available when the `property()` function was originally introduced. However, the decorator syntax was added in Python 2.4, and using `property()` as a decorator has become a popular practice.

Code-Along

This lesson is a code-along, so fork and clone the repo.

NOTE: Remember to run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
pipenv install
pipenv shell
```

Reviewing `property()`

Before we see how to use a decorator to define a property, let's review the original `property()` function.

We'll start with a `Dog` class with attributes for `name` and `breed`.

- `name` must be a string of 1 to 25 characters in length
- `breed` must be a string in the list of approved breeds

We define getter and setter methods for the attributes, then use the `property` function to enforce that the methods are used to access and mutate the attributes:

```
APPROVED_BREEDS = [
    "Mastiff",
    "Chihuahua",
    "Corgi",
    "Shar Pei",
    "Beagle",
    "French Bulldog",
    "Pug",
    "Pointer"
]
```

```
class Dog:
```

```
def __init__(self, name='Fido', breed='Mastiff'):
    self.name = name
    self.breed = breed

def get_name(self):
    return self._name

def set_name(self, name):
    if isinstance(name, str) and 1 <= len(name) <= 25:
        self._name = name.title()
    else:
        raise ValueError(
            "Name must be string between 1 and 25 characters.")

name = property(get_name, set_name)

def get_breed(self):
    return self._breed

def set_breed(self, breed):
    if breed in APPROVED_BREEDS:
        self._breed = breed
    else:
        raise ValueError("Breed must be in list of approved breeds.")

breed = property(get_breed, set_breed)
```

Let's use the debugger to create a few instances of `Dog` .

Type `python lib/debug.py` to start an `ipdb` session.

If we create a `Dog` with a valid name and breed, no exception is thrown:

```
ipdb> Dog("Fido", "Corgi")
<dog.Dog object at 0x1051dc700>
```

However, passing a name that is not a string results in a `ValueError` exception:

```
ipdb> Dog(7, "Pug")
*** ValueError: Name must be string between 1 and 25 characters.
```

A name with an invalid length also throws the exception:

```
ipdb> Dog("Fido the Pug who likes to roll in the mud", "Pug")
*** ValueError: Name must be string between 1 and 25 characters.
```

An exception is thrown if the breed is not in the approved list:

```
ipdb> Dog("Fifi", "Poodle")
*** ValueError: Breed must be in list of approved breeds.
```

Type `exit()` to exit out of the `ipdb` session.

► *Will this code throw an exception?*

```
dog = Dog("Snoopy", "Beagle")
dog.breed = "Beagle, Blue Tick"
```

The lesson repo contains a file `dog_property_test.py` for testing the `Dog` class and its properties.

```
from dog import Dog
import pytest
```

```
class TestDogProperties:
    '''Class Dog in dog.py'''

    def test_name_breed_valid(self):
        '''validates name and breed properties are initialized with valid values'''
        try:
            # No exception should be thrown since name and breed are valid
            dog = Dog("Fido", "Corgi")
        except Exception as exc:
            # The assertion fails if an exception is thrown
            assert False, f'Dog("Fido", "Corgi") raised an exception {exc}'

    def test_name_is_string_valid_length(self):
        '''validates name property is assigned a string between 1 and 25 characters'''
        dog = Dog("Fido", "Corgi")
        with pytest.raises(ValueError):
            dog.name = 7 # not a string
        with pytest.raises(ValueError):
            dog.name = '' # too short
        with pytest.raises(ValueError):
            dog.name = 'Fido the adorable Corgi who likes to steal socks' # too long

    def test_breed_is_approved_breed(self):
        '''validates breed property is in list of approved choices'''
        dog = Dog("Snoopy", "Beagle")
        with pytest.raises(ValueError):
            dog.breed = "Poodle" # not an approved breed
```

Let's run the tests to confirm the current implementation of the `name` and `breed` properties. The tests should pass.

```
pytest -x
```

Function Decorators

We decorate a function by placing the name of the decorator with a leading `@` symbol before the definition of the function you want to decorate:

```
@decorator
def func(a):
    return a
```

This syntax is equivalent to the following:

```
def func(a):
    return a
```

```
func = decorator(func)
```

Using `property()` as a Decorator

We use the `@property` decorator to define a property's getter method. The method should use the public name for the underlying managed attribute, for example `name`.

Delete the `get_name` method:

```
def get_name(self):
    return self._name
```

Add the `name` getter method decorated with `@property` as shown:

```
@property
def name(self):
```

```

    """The name property"""
    return self._name

```

Delete the `set_name` method:

```

def set_name(self, name):
    if isinstance(name, str) and 1 <= len(name) <= 25:
        self._name = name.title()
    else:
        raise ValueError(
            "Name must be string between 1 and 25 characters.")

```

Add the `name` setter method decorated with `@name.setter` as shown:

```

@name.setter
def name(self, name):
    """Name must be a string between 1 and 25 characters in length"""
    if isinstance(name, str) and 1 <= len(name) <= 25:
        self._name = name
    else:
        raise ValueError("Name must be string between 1 and 25 characters." )

```

Finally, you can delete the call to the `property` function as the decorators have defined the getter and setter methods:

```

name = property(get_name, set_name)

```

We'll define the `breed` property in a similar manner. The final version of the `Dog` class is as shown:

```

APPROVED_BREEDS = [
    "Mastiff",

```

```
    "Chihuahua",  
    "Corgi",  
    "Shar Pei",  
    "Beagle",  
    "French Bulldog",  
    "Pug",  
    "Pointer"  
]
```

```
class Dog:  
    def __init__(self, name='Fido', breed='Mastiff'):  
        self.name = name  
        self.breed = breed  
  
    @property  
    def name(self):  
        """The name property"""  
        return self._name  
  
    @name.setter  
    def name(self, name):  
        """Name must be a string between 1 and 25 characters in length"""  
        if isinstance(name, str) and 1 <= len(name) <= 25:  
            self._name = name  
        else:  
            raise ValueError(  
                "Name must be string between 1 and 25 characters."  
            )  
  
    @property  
    def breed(self):  
        """The breed property"""
```



```
        return self._breed

    @breed.setter
    def breed(self, breed):
        """Breed must be in the list of approved breeds"""
        if breed in APPROVED_BREEDS:
            self._breed = breed
        else:
            raise ValueError("Breed must be in list of approved breeds.")
```

Notice that now the method names actually match the attributes `name` and `breed`, and the decorators clearly indicate the purpose of each method. This is considered to be more Pythonic - more intuitive to Python developers in general.

If we need methods for deleting the properties, we would decorate the methods with `@name.deleter` and `@breed.deleter`.

Run the tests to confirm the updated class implementation works.

```
pytest -x
```

You can also use the `ipdb` debugger to instantiate `Dog` objects and attempt to set the `name` and `breed` attributes to valid and invalid values.

Conclusion

Let's summarize the important points when creating properties with the decorator approach:

- The `@property` decorator must decorate the getter method, which should have the same name as the attribute.
- The docstring must go in the getter method.
- The setter and deleter methods must be decorated with the same name as the getter method plus `.setter` and `.deleter`, respectively.

Resources

- [Python Property Decorator](https://www.programiz.com/python-programming/property) ➞ [_ \(https://www.programiz.com/python-programming/property\)](https://www.programiz.com/python-programming/property)
- [Introduction to the Python Property Decorator](https://www.pythontutorial.net/python-oop/python-property-decorator/) ➞ [_ \(https://www.pythontutorial.net/python-oop/python-property-decorator/\)](https://www.pythontutorial.net/python-oop/python-property-decorator/)

This tool needs to be loaded in a new browser window

Load Property Function Decorator : Code-Along (CodeGrade) in a new window